Master Thesis Defense

Design and Implementation of an IoT-based Digital Twin Using ESP32 and Machine Learning

A hardware-aware smart home architecture combining sensing, real-time control, and future-ready Embedded ML/TinyML deployment.

Amirhossein Sadeghi

Supervisor: Dr. Masoud Reza Hashemi

Spring 2026

Smart Home as a Cyber-Physical System

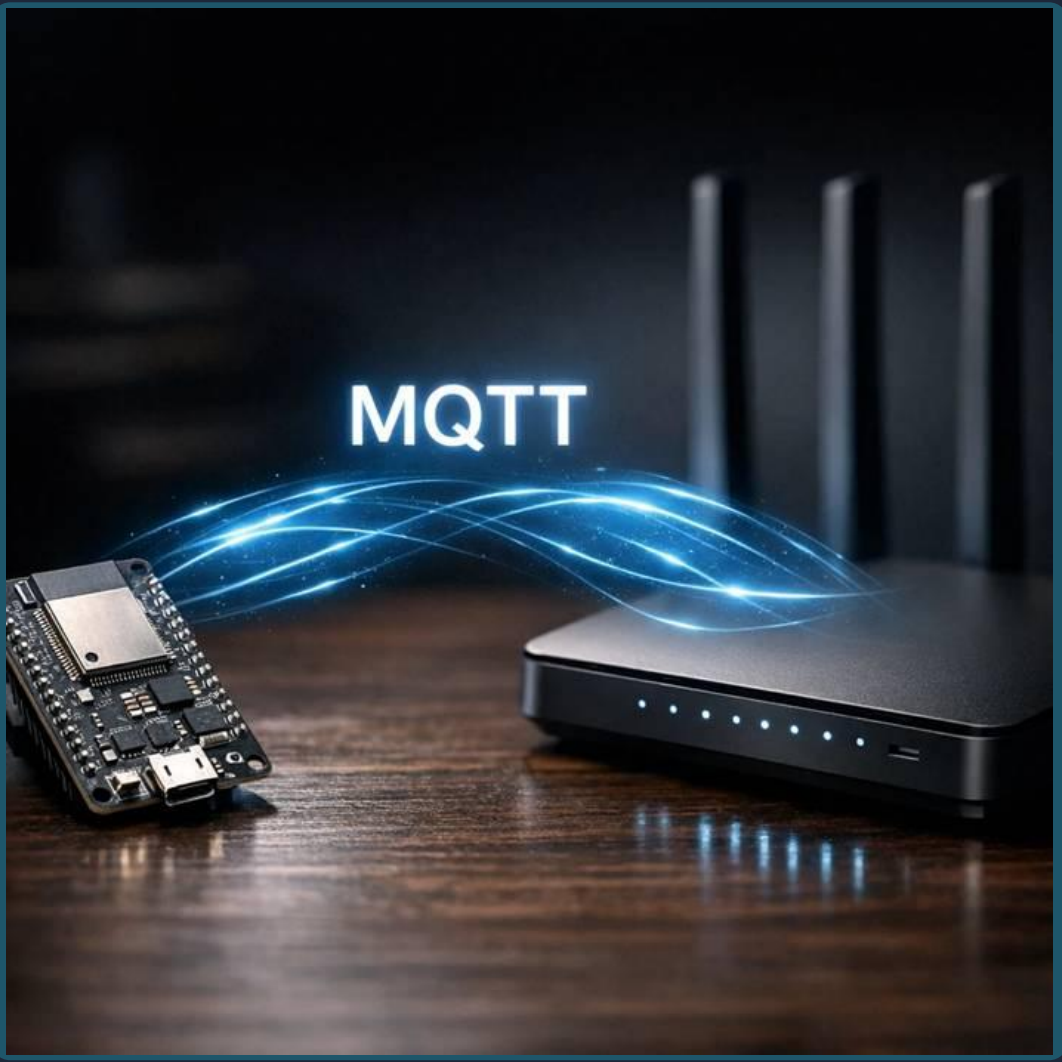
A system-level hardware perspective

- Shift from isolated manual controllers to integrated Cyber-Physical Systems (CPS).
- Distributed embedded nodes connect sensors, actuators, and digital services across the environment.
- Edge computing on local microcontrollers reduces latency and dependence on cloud services.
- Key hardware concerns: communication robustness, power management, and 24/7 reliability.
- This architecture is also suitable for future on-device inference with lightweight Embedded ML.

IoT Networks and MQTT Communication

Reliable messaging for embedded devices

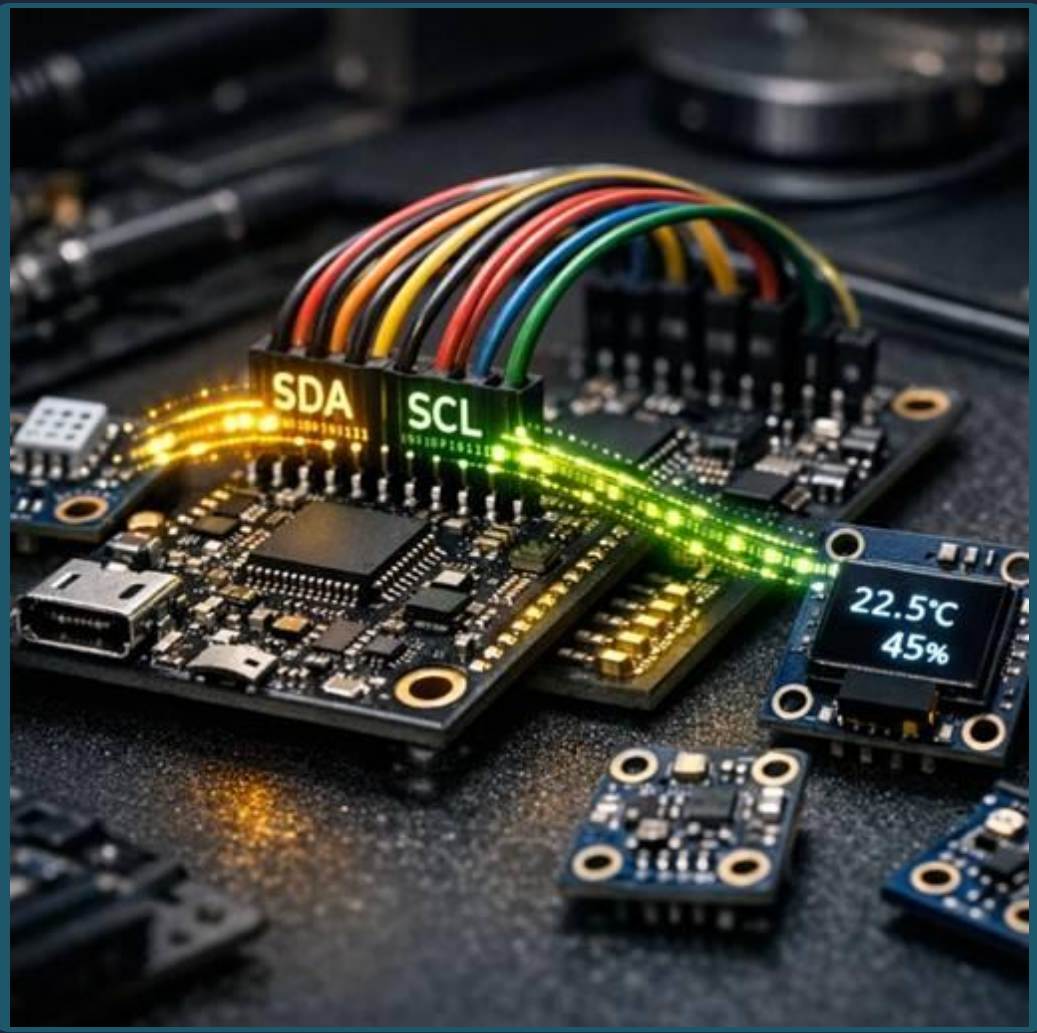
- MQTT provides a lightweight publish/subscribe architecture for real-time embedded communication.
- Sensor readings are transmitted every 3 seconds with QoS 0 to minimize overhead and maximize throughput.
- Network settings are isolated in config.h for cleaner and safer deployment.
- The ESP32 continuously monitors Wi-Fi and MQTT status and performs automatic reconnection in loop().
- The same pipeline can later host TinyML inference either on the edge node or near-edge gateway.



Sensor Network and I2C Integration

Shared-bus sensing with reliability in mind

- A shared I2C bus allows multiple peripherals to be connected through only SDA and SCL on the ESP32.
- This reduces wiring complexity and preserves scarce GPIO resources.
- BME280 measures temperature, humidity, and pressure; BH1750 measures light intensity; VL53L0X provides Time-of-Flight distance sensing.
- Boot-time health checks validate each sensor in setup_sensors before normal operation begins.
- Fault isolation prevents a single failed device from stopping the entire smart-home node.



Digital Twin and Interactive Dashboard

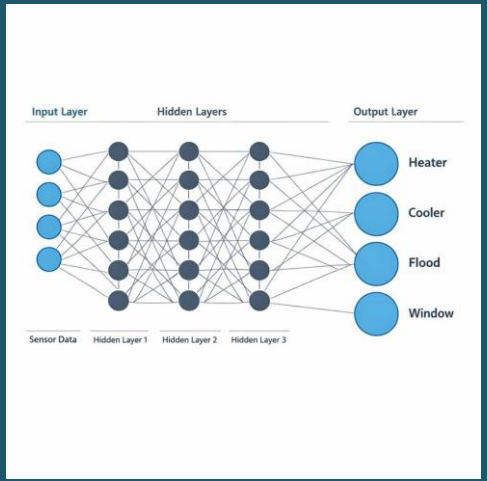
Real-time visualization and closed-loop control

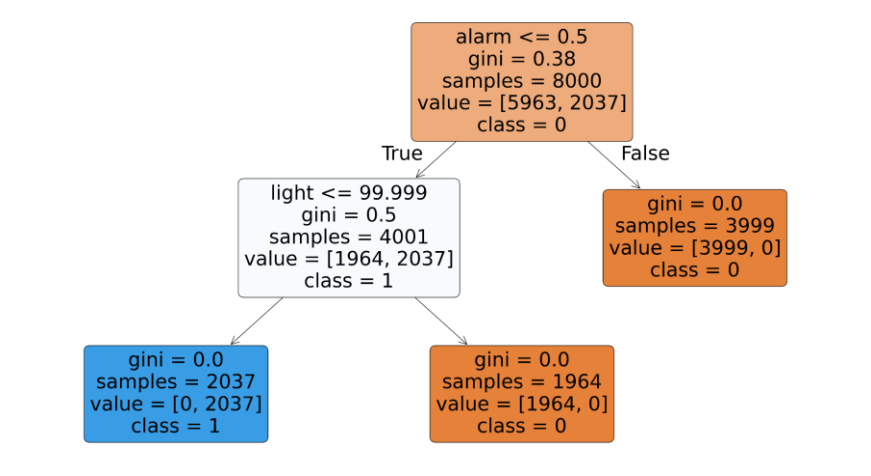
- Node-RED receives ESP32 telemetry from iot/test/sensors and converts it into live gauges and indicators.
- Alarm state is derived from distance measurements and shown immediately on the dashboard.
- Raw sensor data is forwarded to iot/model/input as the input stream for machine learning.
- A Python service processes the features and publishes actuator predictions to iot/model/predictions.
- This clean separation between sensing and intelligence simplifies future migration to edge-deployed ML.

System Intelligence

From reactive control to hardware-aware AI

- The smart home moves beyond rule-based responses toward pattern understanding and need prediction.
- Decision Trees learn optimized control rules directly from real sensor data.
- Neural Networks capture nonlinear relationships for simultaneous multi-actuator control.
- For embedded deployment, lightweight models such as pruned trees or quantized networks are especially attractive.
- A key engineering trade-off is balancing prediction quality against RAM, Flash, and inference latency on ESP32-class hardware.

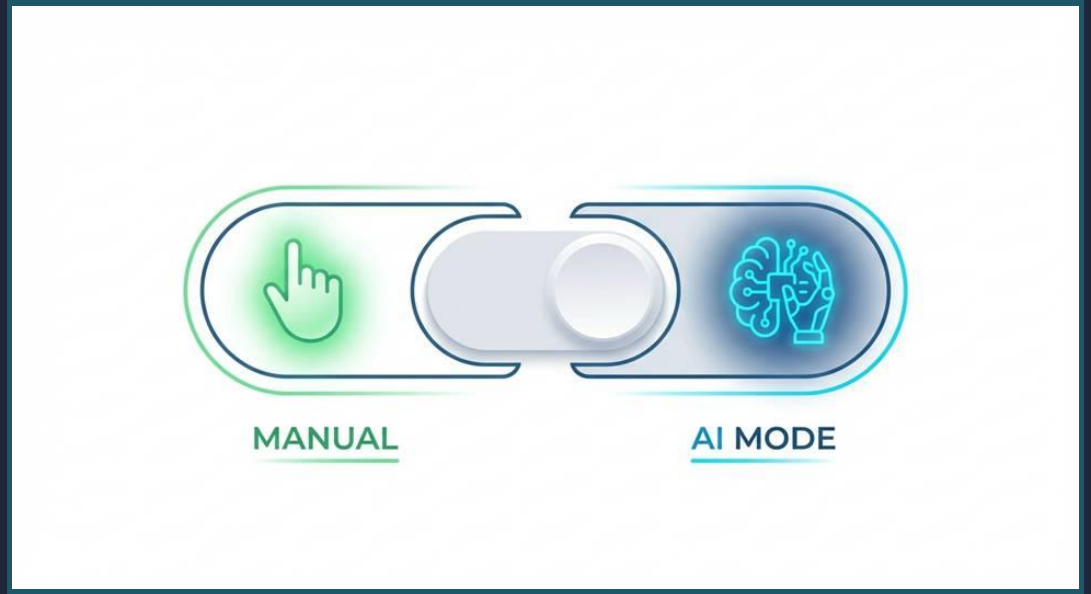




Autonomy and Human-AI Interaction

Switching between operator control and intelligent control

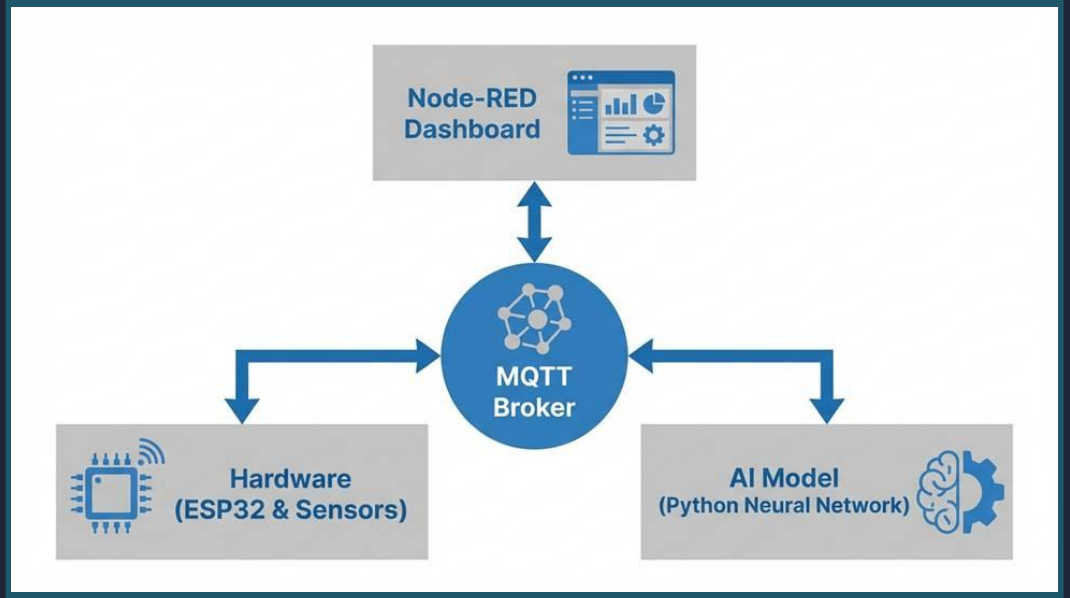
- The system supports instant switching between human control and AI-driven autonomy.
- In Manual/Rule mode, the user directly controls the heater, cooler, pump, window, and other actuators through Node-RED.
- A physical LED indicates mode status: green for manual control and blue for AI mode.
- In AI mode, the neural model takes over actuator decisions while the dashboard dynamically hides manual controls and shows AI predictions.
- For safety, alarm conditions override autonomous commands and force secure behavior.



System-Level Architecture

End-to-end block diagram of the platform

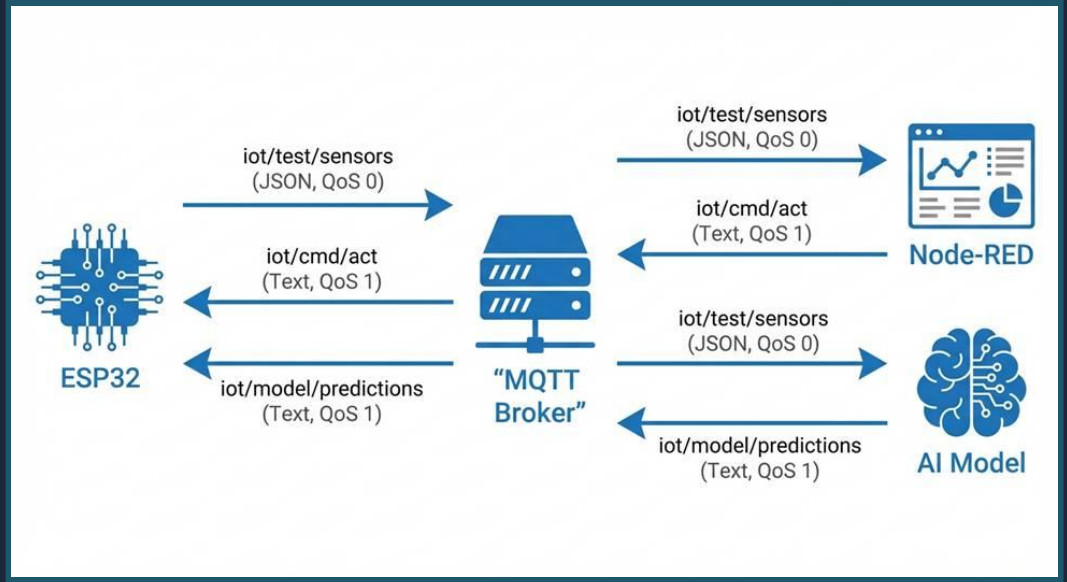
- Edge Layer: ESP32, environmental sensors, and actuators collect physical data and execute commands.
- Node-RED Layer: provides the dashboard, control logic, and AI/manual mode switching.
- AI Layer: a Python service and MLP model process sensor streams and generate autonomous predictions.
- MQTT Broker: the communication backbone that links hardware, dashboard, and AI components in real time.
- Future work can migrate the AI layer closer to the ESP32 using TinyML frameworks and model compression.



MQTT Topics and Message Formats

Communication contracts between modules

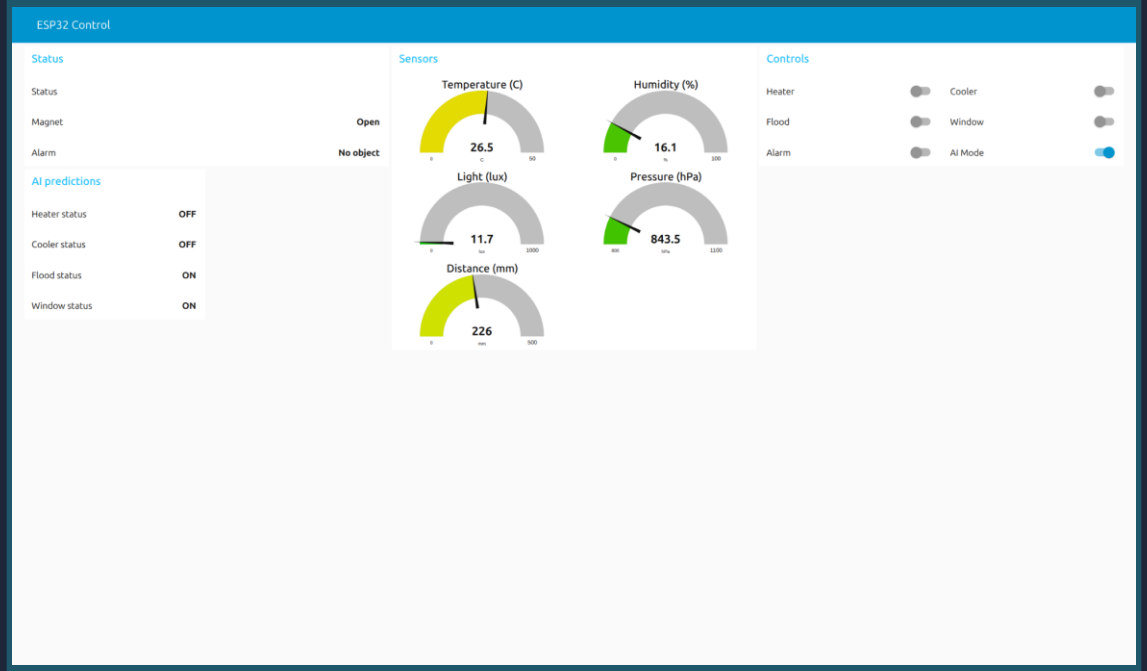
- iot/test/sensors carries JSON telemetry from ESP32 to the broker.
- iot/cmd/act carries manual commands and mode changes from the broker to ESP32.
- iot/model/predictions carries AI-generated control outputs back to the ESP32.
- QoS 0 is used for fast sensor streaming, while QoS 1 is preferred for control messages that must arrive at least once.
- Payloads are deliberately simple: structured JSON for sensing and compact key=value commands such as heater=1 or mode=ai.



Node-RED Dashboard and User Interface

Operational visibility and dual-mode control

- The dashboard shows real-time temperature, humidity, pressure, light, and distance using gauges and charts.
- Users can directly manage actuators in manual mode through a clear control panel.
- A single AI-mode switch transfers control to the neural network model.
- The interface adapts dynamically by revealing the AI prediction panel only when autonomy is enabled.
- Connection state and alarm indicators make system status explicit for the operator.



Key Capabilities and Engineering Achievements

Why this implementation is practically strong

- Fault tolerance: I2C sensor health is checked at startup so a single faulty module does not halt the system.
- Security overriding: presence or intrusion alerts take priority over comfort-oriented actuators.
- Smart energy control: machine learning replaces simple timers and thermostats with data-driven actuation.
- Local edge/fog processing keeps latency low and reduces dependence on external Internet access.
- The modular AI pipeline was intentionally designed so models can later be compressed and deployed directly on embedded hardware.

Conclusion and Future Work

From prototype to publishable embedded intelligence

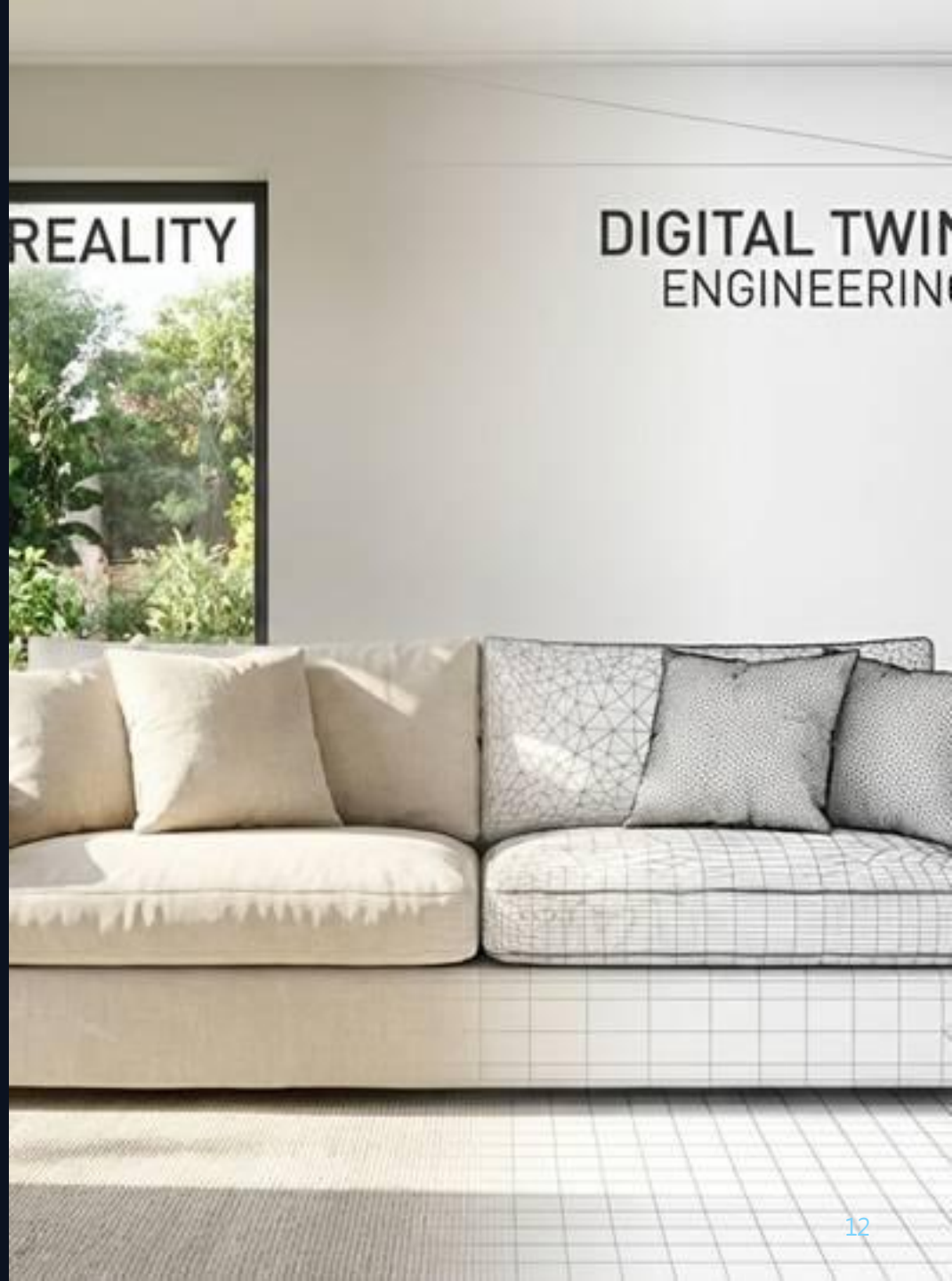
- The project successfully demonstrates a Digital Twin for smart-home equipment using low-cost ESP32 hardware.

- It confirms the feasibility of autonomous local control with practical embedded communication and sensing.

Future Work

- Deploy compressed ML models directly on the ESP32 using TinyML frameworks.

- Apply quantization, pruning, and latency-aware optimization for low-power inference.

- Explore reinforcement learning, continual adaptation, and edge AI vision modules for richer autonomy.

Thank you for your attention.